

The Geometry of Chinese Syntax in Pre-trained Language Models: Structural Probes, Pre-training Objectives, and Control Tasks

中文预训练语言模型的句法几何：结构探针、预训练策略与控制检验

Zhiyang Chen Zhejiang Yongjia High School, Grade 11

Status: final v6 (2026-10-04). Chapter-by-chapter finalization is complete; all numbers come from results/*.json (main-line probes for four models $\times$ 13 layers, mean-pooling ablation for three models, 14 control runs, MDL for four models), automatically summarized into docs/results-tables.md by src/summarize.py; figures are generated into figures/ by src/plot_results.py (the 615 cells of the eight body tables have been checked one by one against the result files, and the Chinese and English versions correspond line by line with identical multisets of number tokens). The bibliography was verified against primary sources (first round 2026-10-02; the RoBERTa entry was added and verified on 2026-10-04). The limitations listed in Section 5 are the items known to the author and kept as such.

Abstract

Do the internal representations of Chinese pre-trained language models (PLMs) encode syntactic structure? This paper systematically applies the structural probe method of Hewitt & Manning (2019) to four models of the same size — three Chinese monolingual models with different pre-training objectives (bert-base-chinese, RoBERTa-wwm-ext, MacBERT) and the multilingual mBERT — examining, layer by layer on the UD Chinese-GSDSimp treebank, the geometric decodability of dependency syntax, and applying control tasks in two directions (selectivity) as well as an MDL information-theoretic test. We find: (1) Chinese syntactic trees can be effectively reconstructed from squared distances after a linear transformation, and the dev UUAS peaks of the three monolingual models **all fall in layer 8** (0.587, 0.587, 0.589), with test peaks of 0.596–0.605, far above the random-tree baseline of 0.096; the curves are hump-shaped, rising first and then falling. (2) The three masking strategies (original MLM, whole-word masking, error-correcting masking) converge to almost the same syntactic geometry: the peak layer is identical, the maximum gap across all layers is 2.6 percentage points, and at the peak layer it is only 0.2 percentage points; replacing the word representation from "first subtoken" with "mean within the word" shifts the absolute values up by about 3 percentage points overall and shifts both the depth and the distance selectivity up in step (layer 8 distance Δ UUAS 0.49 $\rightarrow$ 0.52), while the peak layer and the curve shape are unchanged — the conclusion does not depend on the Chinese-specific character $\leftrightarrow$ word alignment. (3) Multilingual mBERT is not weaker than the monolingual Chinese models under the same pipeline (dev peak 0.604, test 0.619, both higher than the three monolingual models), but its UUAS peak comes earlier, at layer 7, and the distance selectivity and the minimum MDL code length (158.3 kbit) also fall in layer 7 (the three lines agree in direction, each gap being on the order of 1 percentage point); its depth selectivity (+0.095 at layer 8) falls within the range of the monolingual models, showing that this result is not an illusion caused by word-form memorization. (4) Control tests show that the depth probe has negative selectivity in the lower layers (layers 0–4; as low as -0.16) — there the probe relies mainly on word-form/word-type memorization rather than syntax; the point where selectivity turns positive coincides exactly with the syntactic peak at layer 8 ($\Delta = 0.07, 0.07, 0.11$). Under the fixed random-tree control, the distance probe shows strongly positive selectivity in all layers (Δ UUAS 0.28 $\rightarrow$ 0.49), and the control probe's dev correlation is ≈ 0 while its train correlation is positive, showing that the random structure is merely "memorized" and does not enter the representation. (5)

The MDL test independently reproduces the same conclusion: the online code length for the 45 dependency relation labels is lowest at layer 8 (157–161 kbit, compression ratio 3.37–3.45×, test accuracy 0.798–0.801). Three mutually independent lines of evidence (probe metrics, control tasks, information theory) point to the same layer. To our knowledge, this is the first structural probe study of a Chinese PLM family with a complete set of control tests.

Keywords: structural probe; interpretability; Chinese BERT; dependency syntax; control task; minimum description length

1 Introduction

Pre-trained language models perform excellently on various syntax-sensitive tasks, but "high downstream scores" do not amount to "having genuinely learned linguistically meaningful structure internally." Distinguishing the two requires examining the representation itself directly: if some linguistic property really exists in the representation in some geometric form, then a sufficiently simple (linear) readout function should be able to recover it from the representation.

Chinese has independent value on this line of inquiry. Chinese has no morphological inflection, uses characters as the basic unit of writing (and, for the current generation of models, essentially characters as tokens too), and assigns the main grammatical function to word order; conclusions drawn from English — such as the layer-order analysis of Tenney et al. (2019) — cannot simply be carried over. Existing cross-lingual evidence (Chi, Hewitt & Manning 2020) even lists Chinese among the more weakly encoded languages for mBERT. On the other hand, the Chinese PLM family provides a rare, clean control group: bert-base-chinese, RoBERTa-wwm-ext (HFL's Chinese re-training of RoBERTa (Liu et al. 2019), switching to whole-word masking, wwm), and MacBERT (which replaces masked words with synonyms, an error-correcting objective) have **exactly the same size, number of layers, and architecture**, differing only in pre-training objective. This is a natural experiment for studying the causal chain from "training objective → internal structure."

This paper poses three research questions:

- **RQ1 (existence and location):** Can each layer's representation reconstruct the UD Chinese dependency tree via a linear transformation? At which layer is the peak?
- **RQ2 (pre-training strategy and model provenance):** does changing the masking strategy (MLM / wwm / error-correcting) or the language coverage (multilingual mBERT) change the geometry of the syntactic encoding itself, or only the downstream scores?
- **RQ3 (survival under testing):** Do the above findings still hold under the selectivity control and the MDL test?

Contributions:

1. To our knowledge, the first combination of "structural probe + a Chinese monolingual model family + two-way selectivity controls + MDL" applied to Chinese; the closest related work is Zheng & Liu (2023), but it is a single model, uses attention-head/surface-level tasks, and has no structural probe and no controls.
2. We find that the mid-layer syntactic geometry of the three models is highly convergent (same peak layer, a maximum across-layer gap of ≤ 2.6 percentage points and a 0.2-percentage-point gap at the peak layer), suggesting that the syntactic geometry is robust to the pre-training masking strategy; replacing the word representation from "first subtoken" with "mean within the word" leaves the peak layer and the curve shape unchanged while shifting both selectivity measures up in step,

showing that the conclusion also does not depend on the Chinese-specific character→word alignment.

3. Multilingual mBERT is no weaker than the monolingual Chinese models under the same pipeline (dev peak 0.604, test 0.619), but three lines — the UUAS peak, the distance selectivity and the minimum MDL code length — agree in moving its peak to layer 7, which does not conflict with the cross-lingual finding that Chinese is encoded relatively weakly inside mBERT: the two address different levels of the question (4.2b, Section 5).
4. We use control tasks to delimit the credible range of the "syntactic peak": the depth-control selectivity is negative in layers 0–4 (the apparent decodability of the lower layers comes mainly from word-form memory) and turns positive from layer 5 onwards; only the signal above layer 5, and especially at layer 8, survives under both controls.
5. MDL independently reproduces layer 8 with a completely different ruler, the number of bits; the online coding (each block is encoded by a probe trained only on the preceding blocks) leaves no gain in "memorizing the whole table", so the objection of "false positives caused by an over-capacity probe" does not hold.
6. An open-source, reproducible minimal pipeline: all results are obtained on a single laptop CPU (no GPU) in about 30 hours (main-line probes 10.5 h, distance control 15.1 h, depth control 1.4 h, MDL 0.3 h, representation extraction about 2.5 h); the code, all result files and the paper sources are at <https://github.com/Shizuku-keop/chinese-plm-syntax-probe> . Preprint DOI: 10.5281/zenodo.23130689.

2 Related Work

The lineage of structural probes. Hewitt & Manning (2019) propose the distance probe: learn a linear transformation B such that the squared distances between transformed vectors approximate the dependency-tree distances, then reconstruct the tree with a minimum spanning tree and evaluate it with UUAS (as is conventional, punctuation is excluded from their UUAS evaluation — ours includes it, see 3.1 and limitation ⑥ in Section 5). On English Penn Treebank they verify layer by layer that such a transformation really exists for ELMo and BERT-base/large (BERT-large > BERT-base > ELMo) and find that the required rank is surprisingly low (gains stop beyond $k = 64$ –128); their case studies use layer 16 of BERT-large (out of 24) and layer 1 of ELMo. Tenney et al. (2019) use edge probing to show that the layers roughly follow the order of the classical NLP pipeline; Clark et al. (2019) locate attention heads that track dependency relations (the evidence for, and the disputes about, this division of labour across layers are surveyed by Rogers et al. 2020). Kulmizev et al. (2020) compare two annotation styles, UD and surface-syntactic SUD, across 13 languages and find that both BERT and ELMo consistently prefer UD, with the strength of the preference varying across languages and layers — showing that "the structure a probe recovers" depends on the formalism chosen, which is also why this paper fixes a single treebank and annotation scheme and lists the point among its limitations.

Methodological critiques of probing. Hewitt & Liang (2019) point out that a high accuracy may come from a probe memorizing an arbitrary word-form–label mapping, and therefore propose control tasks and selectivity = linguistic-task metric – control-task metric. Voita & Titov (2020) and Pimentel et al. (2020) argue for measuring a representation by its information content (in bits) rather than by accuracy, charging the probe's own complexity to the account (MDL). This paper responds to both critiques: control tasks (3.2) and MDL (3.3). We do **not** perform the Pareto-frontier sweep proposed by Pimentel

et al. (2020) — the probe capacity here is fixed (rank 128, hard-coded; see 3.5.10), and the complexity side is instead reflected by the MDL code length.

Related work on Chinese. CLiMP (Xiang et al. 2021), SLING (Song et al. 2022), and ZhoBLiMP (Liu et al. 2026) are behavioural benchmarks (looking at the model's judgement of sentence-pair probabilities) and do not probe internal representations; Zheng & Liu (2023) performed attention-head probes and surface-task probes on bert-base-chinese, covering 把/被/了/着/过, but without structural probes, without controls, and studying only a single model; Chi et al. (2020) applied structural probes to mBERT on 11 UD treebanks (including Chinese-GSD), and Chinese is one of the more weakly encoded languages, but there is likewise no control test. Our position: we bring "structural probes + model-family comparison + complete controls" to Chinese monolingual models for the first time.

3 Method

3.1 Structural Probes

Distance probe. Let the representation of the i -th word at layer l be $\mathbf{h}_i^{(l)} \in \mathbb{R}^{768}$, and learn a linear transformation $\mathbf{B} \in \mathbb{R}^{128 \times 768}$ such that

$$d_B(i, j) = \|\mathbf{B}(\mathbf{h}_i - \mathbf{h}_j)\|^2$$

approximates the dependency tree distance $d_T(i, j)$ in the treebank (the undirected path length between two words in the dependency tree). The loss is $L = \sum_s \sum_{i < j} |d_T(i, j) - d_B(i, j)|$ (L1, more robust to outliers), optimized with Adam + CosineAnnealing. Note that this is the form of the objective function; in the implementation the loss is **averaged over the valid word pairs within a batch** (`((pred-gold).abs()[pair_mask].mean())`), so the effective step size varies between batches according to the number of elements in the batch — see 3.5.4.

Depth probe. The same form $\|\mathbf{B}\mathbf{h}_i\|^2$ is aligned to node depth (distance to the root), and is used to test the decodability of hierarchical structure.

Evaluation. From the predicted distance matrix we take a minimum spanning tree (MST) to obtain the set of undirected dependency edges, and report UUAS (undirected edge accuracy, i.e. the proportion of the intersection between the predicted edge set and the gold undirected edge set); we also report the Spearman correlation between predicted and gold distances, the depth Spearman, and root accuracy. A caveat about the evaluation convention is in order: our implementation **keeps punctuation words in the graph as ordinary nodes** (punctuation takes part in the MST and its dependency edges are counted in the gold set), excluding only the root's parent edge (the root has no parent edge), see 3.5.4 — this differs from the common practice of "excluding punctuation," which makes the absolute UUAS values higher, so the numbers in this paper are only suitable for longitudinal comparison within this paper's pipeline. UUAS is the hard metric for "can a tree be reconstructed at all," while Spearman is the soft metric for "is the distance matrix monotone and consistent overall"; the two are complementary.

Mathematical note. Expanding gives

$$\|\mathbf{B}(\mathbf{h}_i - \mathbf{h}_j)\|^2 = (\mathbf{h}_i - \mathbf{h}_j)^\top \mathbf{M}(\mathbf{h}_i - \mathbf{h}_j), \quad \mathbf{M} = \mathbf{B}^\top \mathbf{B} \succeq \mathbf{0}, \text{rank}(\mathbf{M}) \leq 128.$$

In other words, the distance probe is learning a **low-rank Mahalanobis distance**: $\mathbf{M}$ being positive semi-definite guarantees symmetry and the triangle inequality — when $\mathbf{M}$ is rank-deficient, d_B is only a **pseudometric** ($d_B(i, j) = 0$ does not imply $\mathbf{h}_i = \mathbf{h}_j$), and only a full-rank $\mathbf{M}$ satisfies the metric axioms strictly; the rank constraint is then equivalent to the hypothesis that the syntax-relevant information is

concentrated in a subspace of at most 128 dimensions (out of the 768-dimensional representation space). If we write M as its spectral decomposition $M = \sum_k \lambda_k \mathbf{u}_k \mathbf{u}_k^\top$, then $\|B(\mathbf{h}_i - \mathbf{h}_j)\|^2$ is the superposition of "projecting the representation difference onto several directions and then weighting the squares by the eigenvalues"; the $\mathbf{u}_k$ that the probe learns are the directions it considers to carry syntactic discriminability. The reason UUAS matters more than Spearman is that it translates "the geometry fits well" into "can a discrete tree be reconstructed from the geometry" — which is the minimal form of a decodability claim.

3.2 Control Tasks and Selectivity

Depth control. Each word type is randomly assigned a "fake depth" that stays fixed over all occurrences of that word type, and the same depth probe is asked to fit this random target. Such a target can only be guessed from word-type identity (word-form/vocabulary information), not computed from syntax. Selectivity is defined as gold depth Spearman – control depth Spearman; a negative value indicates that the probe at that layer is mainly doing word-form memorization.

Distance control. For each sentence, a Prüfer sequence is used to generate a tree with the same length as the true tree but a random structure, and the distance probe is asked to fit it. The difference between random trees and true trees is that a true tree's distances are determined by syntactic structure, whereas a random tree's distances can hardly be recovered except by memorizing word by word. If the probe could "memorize" the random trees of the training sentences, it should fail on dev (because dev consists of entirely new random trees). And that is indeed the story, see 4.3.

3.3 MDL Test

Accuracy has an intrinsic loophole: a probe with more capacity can always squeeze out a higher score, so a high score may only show that the probe is strong. MDL (minimum description length) uses a different ruler and directly counts "how many bits it takes to transmit the labels": if a representation really places the label information "on the surface", then a simple probe can transmit the labels with a very short code; conversely, when the information is absent from the representation, the code length approaches that of a uniform code. The cost of the probe itself is captured by the online coding scheme — each block of labels can only be encoded by a probe trained on the preceding blocks, so extra capacity does not buy a shorter code.

We adopt the online code-length framework of Voita & Titov (2020). All training words are arranged in a fixed random order and cut into prefix blocks (10%, 20%, ..., 100%). The first block has no information available, so it is encoded with a uniform code at a cost of $n_1 \log_2 K$ ($K = 45$ is the number of dependency relation label classes). For block t , we first train a linear classifier (dim $\rightarrow 45$, cross-entropy + Adam, a fixed 5 epochs per prefix) on the data of the previous $t-1$ blocks, then use its output probabilities to encode the gold labels of the current block, at a cost of

$$L = n_1 \log_2 K + \sum_{t \geq 2} \sum_{i \in \text{block } t} -\log_2 p_\theta(\text{gold}_i | \mathbf{h}_i).$$

The smaller L is, the "better" the representation; we report the compression ratio $= n \log_2 K / L$ (the ratio of the uniform code length to the actual code length) as well as the final probe's test accuracy for reference. The key point is that the code length is computed on train, and the probe only ever sees prefix data in each block, so "memorizing the whole table" brings no benefit.

3.4 Experimental Setup

- **Data:** UD Chinese-GSDSimp (simplified), 3,997 / 500 / 500 sentences for train / dev / test; the dependency trees are parsed from CoNLL-U, with 45 dependency relation labels, CC BY-SA 4.0; the treebank is released together with the UD data (the framework is cited as Nivre et al. 2016; it has no standalone paper).
- **Models:** four models of the same size (~102M parameters, a 12-layer Transformer, 13 extraction points including the input layer): the Chinese monolingual `bert-base-chinese` (Devlin et al. 2019), `hfl/chinese-roberta-wwm-ext` (Cui et al. 2021) and `hfl/chinese-macbert-base` (Cui et al. 2020), plus the multilingual mBERT (`bert-base-multilingual-cased`, also from Devlin et al. 2019). All are loaded from local directories with **frozen** weights; only their representations are read.
- **Chinese-specific handling and ablation:** UD is annotated by word while Chinese BERT segments by character, so alignment is needed; the main experiment takes the first-subtoken representation of each word (`--pooling first`, with 0 alignment failures). To test the effect of this choice, we run an additional **mean pooling ablation** (averaging all subtokens within a word, `--pooling mean`): the main-line probes, depth control, distance control, and MDL are re-run for the three monolingual models.
- **Hyperparameters:** distance/depth probes rank = 128, epochs = 20, lr = 1e-3, batch_size = 32, seed = 0; MDL probe lr = 1e-3, 5 epochs per prefix, batch_size = 1024.
- **Compute and runtime:** a single laptop CPU (no GPU). The main-line probe takes about 88 minutes per model (13 layers), depth control about 12 minutes per model, distance control about 80 minutes per model, and MDL about 3 minutes per model; representation extraction takes about 20–28 minutes per "model × pooling" combination, about 2.5 hours for all seven. Summing the seconds field of `results/*.json`, all the results in this repository take about 30 hours in total (main-line probes 10.5 h, distance control 15.1 h, depth control 1.4 h, MDL 0.3 h, plus about 2.5 h of representation extraction).
- **Acceptance gates (smoke-test self-checks):** ① number of parsed sentences 3,997 / 500 / 500; ② character↔word alignment mismatches 0 (the "no-subtoken-coverage fallback" occurs 1 time on an individual word, see 3.5.3); ③ the overfitting gate `--overfit64`: train and evaluate a distance probe simultaneously on the first 64 training sentences, requiring distance Spearman > 0.9 — measured 0.9455 at layer 7, OVERFIT GATE: PASS (log `results/overfit64_bert.log`). This gate verifies that the optimization procedure can indeed fit a given signal, and has nothing to do with representation quality; ④ UUAS falls in a reasonable range (cf. `results/baselines.json`: random tree 0.096, linear chain 0.426) — this item is a manual check, with no automatic assertion in the code. The full list and form of all acceptance gates are given in 3.5.9.

3.5 Implementation

This section explains the code implementation that supports all the results of Sections 3.1–3.4. All scripts are located in `src/` (13 Python files, including a forward-pass-only smoke test `smoke_test.py`), along with four batch-driving shell scripts; all commands are run with the repository root as the working directory, input and output paths are hard-coded as relative paths in the code (e.g. `DATA_DIR = Path("data/ud_gdsimp")`, `OUT_DIR = Path("results/representations")`), and except for `download_model.py`, which explicitly accesses ModelScope, the remaining scripts do not go online (the models and treebank must first be placed locally).

3.5.1 Module Map and Data Flow

The complete chain is "CoNLL-U → representation tensors → probe metrics → tables and figures," where the stages are coupled to one another only through files on disk (`treebank.py` is the only parsing library imported in multiple places):

Stage	Script	Input	Output
Treebank parsing (library)	<code>src/treebank.py</code>	<code>data/ud_gsdsimp/zh_gsdsimp-ud-{train,dev,test}.conllu</code>	list of sentences in memory (not written to disk)
Baselines	<code>src/baselines.py</code>	the three conllu files	<code>results/baselines.json</code>
Model download	<code>src/download_model.py</code> <id>	ModelScope direct link	<code>models/<name>/{config.json,vocab.txt,tokenizer*,model.safetensors}</code>
Representation extraction	<code>src/extract.py</code> <code>models/<name> [--pooling first mean]</code>	conllu + local model	<code>results/representations/<name>/{train,dev,test}.npz</code>
Main-line probe	<code>src/run_first_probe.py</code> <code>--model <name></code>	conllu + three npz files	<code>results/first_probe_<name>.json</code>
Depth control	<code>src/run_control.py --control depth --model <name></code>	same as above	<code>results/control_depth_<name>.json</code>
Distance control	<code>src/run_control.py --control distance --model <name></code>	same as above	<code>results/control_distance_<name>.json</code>
MDL	<code>src/run_md1.py --model <name></code>	conllu + npz	<code>results/md1_<name>.json</code>
Summary tables	<code>src/summarize.py</code>	all of the above JSON	<code>docs/results-tables.md</code>
Plotting	<code>src/plot_results.py</code>	all of the above JSON	<code>figures/fig1_syntax_geometry.png ... fig4_pooling_ablation.png</code>

Key connection points: `extract.py` takes only words and # text (for alignment) from the treebank, while distances/depths/dependency labels are always computed on the fly by `treebank.parse_treebank`; the probe scripts open the conllu and the npz at the same time and require the two to be **identical sentence by sentence in sentence order and word order** — this premise is guaranteed by a word-count assertion (3.5.9). Mean pooling introduces no new script: `extract.py` writes the output directory name as `<model directory name>_mean`, and subsequent scripts reuse the same code path via `--model bert-base-chinese_mean`, the two sets of results never overwriting each other. Run logs are redirected to `results/*.log` by the shell scripts; the scripts themselves do not write logs; `run_md1_all.sh`, `run_extras.sh`, `run_extras_resume.sh`, and `run_distance_means.sh` are the driver scripts actually used to produce this repository's results in batches (the latter three also back up `results/*.json` to the D drive); `run_distance_means.sh` does exactly one thing — it runs the distance control for the mean pooling of the three monolingual models in series, and then automatically calls `summarize.py` and `plot_results.py`.

3.5.2 Data Layer

CoNLL-U parsing (`treebank.py`). Reading is streamed via `conllu.parse_incr`; `words[ i ] = token["form"]` (the word form, not the lemma — the two often differ in Chinese treebanks, e.g. the lemma of "他们" is "他"). The parent is taken as `heads = token["head"] - 1`, converting UD's 1-based column to 0-based, so the root's `head = 0` becomes `-1`. The distance matrix is computed by `_distances_and_depths`: first build an undirected adjacency list from `h ≥ 0` (adding each edge in both directions), then run one BFS from each starting point (`collections.deque`), obtaining an (n, n) undirected shortest-path matrix, i.e. the path length between two words in the dependency tree; for depth, the unique root is located by `heads < 0` and one more BFS gives the number of hops from each word to the root (the root being 0). **Punctuation is not removed:** the function name and docstring explicitly say `*_incl_punct`, and

punctuation words are ordinary tokens in UD (they have a head and `deprel = punct`), so they enter the index, the distance matrix, the depth vector, and all subsequent evaluation (measured: 13,627 punctuation words out of 98,614 in train). The **root** is likewise treated as an ordinary node, except that it has no parent edge and its depth is always 0, so it does not enter the undirected edge set. Multi-word tokens (of the form 1-2) and empty nodes (of the form 3.1) occur 0 times each in the three files, and the code has no branch for them either — parsing relies entirely on conllu's default iteration (no explicit handling of multi-word tokens/empty nodes appears in the code). The parse result also carries an assertion: all elements of the distance matrix are ≥ 0 , i.e. "the tree should be connected." Each sentence also stores `text = sent.metadata["text"]` for character-level alignment (measured: all 4,997 sentences have this field).

Baselines (baselines.py). The linear chain sets the edge set to $(i, i + 1) : 0 \leq i < n - 1$ and takes the predicted distances directly as $|i - j|$ (the last word naturally becomes the endpoint of the chain; the code does not explicitly declare a root). The random tree generates `N_RANDOM = 10` uniformly random labelled trees per sentence: `_random_tree_edges` uses a Prüfer sequence (`rng.integers(0, n, size=n-2)` plus a degree table, repeatedly taking "the leaf with the smallest index" to decode), then `scipy.sparse.csgraph.shortest_path(..., directed=False)` gives the distance matrix. The 10 trees within a sentence each have their UAS/Spearman computed first and are then averaged, after which the average is taken over sentences; the evaluation convention is exactly the same as in `probe.py` (skip `n < 2`, undirected edges, upper-triangular Spearman, NaNs removed). The seed is fixed at 0, and `np.random.default_rng(0)` is re-initialized separately for each split.

3.5.3 Representation Layer: Extraction and Chinese Character→Word Alignment

The flow of `extract.py` is "tokenize the whole batch → forward the whole batch → align sentence by sentence → pool word by word → write the npz."

- **Tokenization and batched forward pass:** `AutoTokenizer.from_pretrained(<local directory>)`, a batch of 16 sentences (`--batch-size`, default 16) calls `tok(texts, padding=True, return_tensors="pt", return_offsets_mapping=True)` ONCE; **truncation is not passed** (truncation is not enabled in the code; the longest sentence in this treebank is 111 words / 182 characters, far below 512, so no length problem was triggered, but the code itself has no truncation or chunking fallback). The model uses `AutoModel.from_pretrained(...).eval()` (not `ForMaskedLM`, hence there are no `lm_head` logits), and the entire `extract_split` function is decorated with `@torch.no_grad()`; the weights are never updated — the code does not explicitly write `requires_grad_(False)`; freezing is achieved by "forward only, no optimizer built."
- **13 extraction points:** `output_hidden_states=True` returns 13 tensors, `n_layers = model.config.num_hidden_layers + 1`. The keys are named `layer_0 ... layer_12`, where `layer_0` is the output of the embedding layer (word embeddings + position/segment embeddings + LayerNorm) and `layer_1...12` are the layer-by-layer outputs of the 12 Transformer layers.
- **Alignment:** `align_words` matches word forms in order on `# text`, first trying `text.startswith(w, cursor)`, then, on failure, skipping whitespace and trying again, and on further failure using `text.find(w, cursor)` to search the whole text and counting it as a "mismatch recovered later"; if it still fails, a `ValueError` is raised (not silently skipped). The subtoken mapping is done by `word_token_indices`: take all non-special subtokens that **overlap** the word form's character interval $[w_s, w_e)$; special symbols are given a $(0, 0)$ offset by the tokenizer, and `[CLS]/[SEP]/[PAD]` are excluded from pooling on that basis.

- **Two poolings:** `--pooling first` (default) takes `ix[0]` for each word, i.e. the first overlapping subtoken — for the character-segmenting BERT family this is the "first character of the word"; `--pooling mean` takes `h[b, ix].mean(0)`, i.e. the mean of all subtokens within the word.
- **Storage:** `layers[li][row:row+n] = word_h.half().numpy()`, i.e. written to disk as float16, and finally `np.savez(dest / f"{split}.npz", **out)` (savez does not compress). The npz keys are `layer_0...` `layer_12` (shape (total words, 768), float16) and `lengths` (shape (n_sents,), int32). The semantics of `lengths` are "split the row dimension by sentence," and `lengths.sum()` must equal the total number of words in the treebank; sentence splitting is done by `probe.split_sentences` using `np.cumsum`.
- **Trigger conditions for the two fallbacks:** ① "mismatch recovered later" — the word form cannot be matched at the current position (including after skipping whitespace) but can be located further on in `# text` (e.g. an inconsistency between inter-word whitespace and a missing `SpaceAfter`); ② "no-subtoken-coverage fallback" — a word's character interval does not overlap any non-special subtoken (the word form is stripped by the tokenizer's normalizer, e.g. combining diacritics), in which case it falls back to the first non-special subtoken after that word, and if there is no subtoken after it, to the last non-special subtoken of the whole sentence. Both counts are only printed (失配再找回: N 词, 无子词覆盖回退: M 词), not written into the npz and not asserted. In the existing logs "mismatch recovered later" is 0 everywhere (0 cases); the zero-coverage fallback is 1 word each on the train split of MacBERT (first and mean) and RoBERTa-wwm-ext (mean), and 0 elsewhere (the extraction log for RoBERTa's first-subtoken pooling is not in the repository, and this is not reflected in the code).

3.5.4 Probe Layer

Network architecture. `DistanceProbe` and `DepthProbe` both have only one parameter, $B \in \mathbb{R}^{\text{rank} \times 768}$, initialized with `torch.randn(rank, dim) * 0.01`, **with no bias**. The forward pass first uses `torch.einsum("rd,bnd->bnr", B, h)` to project the words into rank dimensions; the distance probe then sums $(u_i - u_j)^2$ over the rank dimensions — strictly equivalent to $\|B(h_i - h_j)\|^2$ (by linearity), but avoiding the construction of n^2 difference vectors in 768 dimensions; the depth probe takes $\|u_i\|^2$. Rank 128 appears in exactly one place: the **number of rows** of B is 128, hence the $M = B^T B$ in $\|B(h_i - h_j)\|^2 = (h_i - h_j)^T M (h_i - h_j)$ has rank at most 128.

Loss. Strict L1, averaged over the valid elements in the batch: the distance probe uses `loss = (pred - gold).abs()[pair_mask].mean()`, where `pair_mask = mask @ mask` (the mask is built from the true sentence lengths in the batch, with padding positions excluded); this covers all valid **ordered** word pairs in the batch, including the diagonal elements (i, i) — for which both gold and prediction are 0, so they only dilute the mean. The depth probe uses `(pred - gold).abs()[mask].mean()`. The loss is therefore "the mean over valid word pairs (valid words) within a batch," not apportioned by sentence: the weight of a batch containing long sentences versus one containing short sentences is determined by the number of elements in the batch.

Batching and optimization. `make_batches` reshuffles the sentence order each epoch with `np.random.default_rng(seed).permutation`, then does greedy bin packing: at most `batch_size = 32` sentences per batch, and `n_max^2 × number of sentences ≤ pair_budget = 120000` within a batch (using a quadratic estimate for memory), so long sentences automatically land in small batches. The optimizer is `torch.optim.Adam(lr = 1e-3)` with `CosineAnnealingLR(T_max = epochs = 20)`; inside `train_probe`, `torch.manual_seed(seed)` is called first and then `default_rng(seed)` is created. The representations

themselves are constants (a float32 tensor obtained from `torch.from_numpy`), and gradients flow only through `B`, so there is no path for "the representation adapting to the probe."

Evaluation. `eval_distance_probe` computes sentence by sentence, skipping $n < 2$: the predicted distance matrix is fed to `scipy.sparse.csgraph.minimum_spanning_tree`, and the `(row, col)` of `tocoo()` is taken as the predicted undirected edge set; the gold undirected edge set is built from `heads ≥ 0`. On punctuation and the root: **the code does not remove punctuation** — punctuation words participate in the MST (they can become internal nodes), and their dependency edges are counted in the gold set as well; only the root (`head = -1`) produces no gold edge, so the denominator is always the sentence length minus one; if the root is predicted as a child of some word, that edge is not in the gold set and directly shrinks the intersection. $UUAS = |\text{predicted edges} \cap \text{gold edges}| / |\text{gold edges}|$, averaged over sentences. Spearman takes the predicted and gold distances on the upper triangle ($i < j$) of each sentence via `scipy.stats.spearmanr(...).statistic`, averaged over sentences with NaNs removed. `eval_depth_probe` is analogous: `spearmanr(predicted depth, gold depth)` per sentence (all words, including punctuation), averaged over sentences; root accuracy is the proportion of $\arg \min(\text{pred}) = \arg \min(\text{gold})$ (the word with the smallest predicted depth should be the root). The main-line script trains two independent probes per layer, and resets `torch.manual_seed(args.seed)` before constructing each probe, so both share the same seed.

3.5.5 Control Tasks

Random fake depth. `depth_control_labels` first takes the maximum depth `max_depth` over all sentences in the **first key** of `splits` (`run_control.py` builds the dictionary in the order train/dev/test, so this is train), then maintains a "word form $\rightarrow$ integer" table: the first time a word form is encountered, an integer is drawn uniformly with `np.random.default_rng(seed).integers(0, max_depth + 1)` and fixed permanently. Because iteration proceeds sentence by sentence and word by word in split order, the order in which the random stream is consumed is entirely determined by the order of splits, so the result is deterministic; and because the table is shared across splits, word types in dev/test not seen in train also get a fixed label. Note that the solvability of this target comes from "word-type identity" rather than syntax, which is exactly the point of the control task.

Random trees. `distance_control_trees` generates, for each sentence, a uniformly random labelled tree from a Prüfer sequence via `_random_heads(n, rng)`: $n = 1$ and $n = 2$ are special-cased, decoding takes the leaf with the smallest index at each step, and finally the two remaining degree-1 nodes are connected; then a BFS starting from word 0 orients the tree to give 0-based parents (**the root is fixed to word 0**, independent of the true tree's root position). The distance matrix is computed with `scipy.sparse.csgraph.shortest_path(..., directed=False)`. The number of nodes, sentence lengths, and word order are exactly the same as for the true tree, the only difference being the topology; the random trees of train/dev/test are mutually independent (the same rng stream drawn in split and sentence order), with determinism guaranteed by `seed = 0`.

Parts shared with the main experiment. Both control lines directly reuse `probe.py`'s `DistanceProbe/DepthProbe`, `train_probe`, `eval_distance_probe/eval_depth_probe`, as well as the data loading and assertions of `run_first_probe.load_split`, and the hyperparameters are exactly the same (rank 128 / epochs 20 / lr 1e-3 / batch_size 32 / seed 0); only the "training target" is replaced (true tree/true depth $\rightarrow$ random tree/fake depth). At the end of each layer, `run_control.py` prints the layer's selectivity = gold metric – control metric as a manual check if `results/first_probe_<model>.json` exists; the selectivity in the summary tables is computed on the fly by `summarize.py` from the two JSON files.

3.5.6 MDL Test

Labels. `run_md1.deprel_labels` builds a vocabulary from the sorted, deduplicated set of labels of train (45 classes, including Chinese-specific subtypes such as `case:loc`), and flattens it into indices sentence by sentence; test uses the same vocabulary, and out-of-vocabulary labels are mapped to -1, while the trained classifier never outputs -1, so such words are necessarily wrong in the reference accuracy (measured: the number of unseen test labels is 0 for all four models; this branch is only a safety net and never actually triggers).

Prefix splitting. `PORTIONS = [0.1, 0.2, ..., 1.0];` `order = np.random.default_rng(seed).permutation(len(X))` rearranges all training words in this **fixed random order**, and then `bounds = [0] + [round(t × N)]` is cut ($N = 98,614$). The same seed makes all layers use the same permutation, so layers are comparable.

Code-length accumulation. The first block `[0, bounds[1])` has no information available, so a uniform code is used at a cost of $(\text{bounds}[1] - \text{bounds}[0]) \times \log_2(K)$; measured with $K = 45$, this block is 54,156 bit, and the full uniform code length is `uniform_bits = N × log2 K = 541,574 bit`. Then for $k = 1 \dots 9$: a `torch.nn.Linear(768, 45)` (with bias by default) is trained on **all** the preceding data `Xp[:bounds[k]]`, with cross-entropy loss, Adam optimizer ($\text{lr} = 1\text{e-}3$), a fixed 5 epochs per prefix, `batch_size = 1024` (no early stopping, no validation set, dev unused); its `log_softmax` is then used to accumulate $-\log_2 p(\text{gold})$ over the gold labels of the current block, implemented as `nll = -logp[range(len(block)), y_block].sum()`; `bits += float(nll) / float(np.log(2))` (`log_softmax` is natural logarithm, divided by **ln 2** to convert to bits). The encoding probabilities for the last block (0.9→1.0) come from the probe trained on only the first 90% of the data.

Reporting convention. The total code length `bits` is accumulated on train only; the compression ratio is `compression = uniform_bits / bits`; we also report the last probe's test accuracy `test_acc` for reference (accuracy computes the argmax hit rate in batches of 8192). Each call to `train_classifier` first calls `torch.manual_seed(seed)` and then `default_rng(seed)`, so the probe initialization and the in-batch shuffling order are exactly the same for every prefix, and only the data differs.

uniform_selftest. Running `python src/md1.py` directly executes this self-check: on synthetic data with $n = 2000$, $\text{dim} = 16$, $k = 7$, the training function is replaced by a "zero-weight linear layer" (whose output is strictly uniform), verifying that the relative error between the online code length and $n \log_2 k$ is $< 1\text{e-}3$, printing `UNIFORM GATE: PASS/FAIL` and distinguishing the two by exit code. It verifies that the **code-length accumulation formula and the bit conversion** are indeed equivalent in a case where they should be equivalent to a uniform code, and does not involve real representations.

3.5.7 Engineering Robustness and Reproducibility

- **Per-layer checkpoint + atomic replacement:** the three experiment scripts (`run_first_probe.py`, `run_control.py`, `run_md1.py`) all define `checkpoint()`, which writes "metadata + all completed rows" to `<dest>.tmp` after each completed layer and then overwrites the official file with `tmp.replace(dest)` — a same-directory replacement, so an interruption never leaves half a JSON behind.
- **Checkpoint/resume:** at startup, if the target JSON already exists, the set of completed layer numbers is read from `layers`, and the main loop simply does `if layer in done: continue;` `run_control.py / run_md1.py` additionally have a `--layers` argument to run only specified layers for a quick self-check.

- **Random seeds:** `torch.manual_seed(seed)` (probe/classifier initialization and the torch-side batch order) + `np.random.default_rng(seed)` (sentence-order shuffling, random word-type depths, Prüfer sequences), with `seed = 0` by default, overridable on the command line.
- **CPU multi-threading determinism:** there is no `torch.set_num_threads`, `OMP_NUM_THREADS`, or `torch.use_deterministic_algorithms` setting anywhere under `src/`, nor is `PYTHONHASHSEED` fixed — that is, **no special settings were made**, and bitwise reproducibility depends on the determinism of the CPU operators themselves.
- **fp16 and size:** the npz is written to disk as float16 (converted uniformly with `.astype(np.float32)` during training). A single file's size = number of words $\times$ 13 layers $\times$ 768 $\times$ 2 bytes: under `results/representations/bert-base-chinese/`, `train.npz` $\approx$ 1.97 GB (98,614 words), `dev.npz` $\approx$ 241 MB, `test.npz` $\approx$ 229 MB; the 7 representation directories total about 16.6 GB (the code has no comment explaining the reason for choosing fp16; the direct effect is to halve the disk footprint).
- **Order of magnitude of runtimes** (reading the `seconds` field of each JSON, `bert-base-chinese`): main line, 13 layers in total, 5,261.5 s $\approx$ 88 min (402–408 s per layer); depth control 716.5 s $\approx$ 12 min (53–67 s per layer); distance control 4,818.4 s $\approx$ 80 min (369–374 s per layer); MDL 172.0 s $\approx$ 3 min (13–15 s per layer); mBERT main line 5,414.6 s, the same order of magnitude as the monolingual models.

3.5.8 Reproduction Steps and Environment

The following commands are executed in order under Git Bash in the repository root (Windows path style, with `PYTHONIOENCODING=utf-8` used for Chinese output):

```

# 0) environment
python -m venv .venv
./venv/Scripts/python -m pip install -r requirements.txt
./venv/Scripts/python -m pip install torch --index-url https://download.pytorch.org/whl/cpu

# 1) treebank (jsDelivr channel, see docs/feasibility.md; there is no download script in the repository, and the paths follow the
actual file names in the data directory)
mkdir -p data/ud_gsdsimp
for s in train dev test; do
    curl -L -o "data/ud_gsdsimp/zh_gsdsimp-ud-$s.conllu" \
        "https://cdn.jsdelivr.net/gh/UniversalDependencies/UD_Chinese-GSDSimp@master/zh_gsdsimp-ud-$s.conllu"
done
./venv/Scripts/python src/treebank.py data/ud_gsdsimp/zh_gsdsimp-ud-train.conllu # should print 3997 sentences

# 2) models (ModelScope direct link -> models/<name>/)
./venv/Scripts/python src/download_model.py AI-ModelScope/bert-base-chinese
./venv/Scripts/python src/download_model.py dienstag/chinese-roberta-wwm-ext
./venv/Scripts/python src/download_model.py dienstag/chinese-macbert-base
./venv/Scripts/python src/download_model.py AI-ModelScope/bert-base-multilingual-cased

# 3) representation extraction: four models (first-subtoken pooling) + three monolingual models (mean pooling)
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    ./venv/Scripts/python src/extract.py "models/$m" --pooling first
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    ./venv/Scripts/python src/extract.py "models/$m" --pooling mean
done

# 4) overfitting gate + baselines
./venv/Scripts/python src/run_first_probe.py --overfit64 --model bert-base-chinese --layer 7
./venv/Scripts/python src/baselines.py

# 5) main-line probes (four models first-subtoken + three models mean)
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    ./venv/Scripts/python src/run_first_probe.py --model "$m"
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    ./venv/Scripts/python src/run_first_probe.py --model "${m}_mean"
done

# 6) control experiments (depth + distance; four models with first-subtoken pooling + three monolingual models with mean pooling)
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    for c in depth distance; do
        ./venv/Scripts/python src/run_control.py --control "$c" --model "$m"
    done
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    for c in depth distance; do
        ./venv/Scripts/python src/run_control.py --control "$c" --model "${m}_mean"
    done
done

# 7) MDL (including the self-check)
./venv/Scripts/python src/mdl.py
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base bert-base-multilingual-cased; do
    ./venv/Scripts/python src/run_md1.py --model "$m"
done
for m in bert-base-chinese chinese-roberta-wwm-ext chinese-macbert-base; do
    ./venv/Scripts/python src/run_md1.py --model "${m}_mean"
done

# 8) summarization and plotting (reads results/*.json only)
./venv/Scripts/python src/summarize.py # -> docs/results-tables.md
./venv/Scripts/python src/plot_results.py # -> figures/fig1..4.png

```

Four batch driver scripts are actually used by this repository: `bash run_md1_all.sh` (three models' MDL in series), `bash run_extras.sh` (mBERT comparison + mean pooling ablation, 9 stages in total; after an

interruption, use `bash run_extras_resume.sh` to resume from stage 7), and `bash run_distance_means.sh` (distance control for the mean pooling of the three monolingual models, with per-layer resume from checkpoints, followed by automatic summarization and plotting on completion). All four scripts append their output to `results/*.log`.

Environment (from `./venv/Scripts/python -m pip freeze`): Python 3.13.9; torch 2.14.1+cpu; transformers 5.18.0; tokenizers 0.23.2; conllu 6.0.0; numpy 2.5.3; scipy 1.18.1; matplotlib 3.11.2; safetensors 0.8.0; huggingface_hub 1.33.0. The hardware is a single laptop CPU (no GPU, CUDA unused).

3.5.9 Acceptance Gates (Smoke-Test Self-Checks)

Check	Location	Criterion and form
Number of parsed sentences 3,997 / 500 / 500	The main flow of <code>src/extract.py</code> prints <code>[train] 3997 句, 98614 词 per split</code> ; running <code>src/treebank.py</code> directly also prints it	printed item, not an assertion
npz word count matches the treebank	<code>assert out["lengths"].sum() == n_words</code> in <code>src/extract.py</code> ; asserted once more in <code>src/run_first_probe.load_split</code> (shared by the three experiment scripts)	assertion, aborts on failure
Alignment mismatch / zero-coverage counts	<code>src/extract.py</code> prints the two counts	printed item, not written to disk, not asserted
Conservation of word count within a batch	<code>assert row == total_words</code> in <code>extract_split</code>	assertion
Tree connectivity	<code>treebank._distances_and_depths</code> asserts that all BFS distances are ≥ 0 ; <code>control_tasks._random_heads</code> asserts that the random tree is connected	assertion
--overfit64 gate	<code>src/run_first_probe.py::run_overfit64</code>	On the first 64 training sentences, train and evaluate a distance probe at the same layer (<code>batch_size = 8</code> , layer defaults to 7); only if <code>Spearman > 0.9</code> does it print <code>OVERFIT GATE: PASS</code> and exit with code 0, otherwise FAIL with exit code 1. It verifies that "the optimization procedure can indeed fit a given signal," independent of representation quality
MDL uniform self-check	<code>src/mdl.py::uniform_selftest</code> (python <code>src/mdl.py</code>)	Under a uniform probe, relative error between the online code length and the uniform code length $< 1e-3 \rightarrow$ <code>UNIFORM GATE: PASS</code> , otherwise FAIL with exit code 1
UUAS falls in a reasonable range	no automatic assertion	The reference is <code>results/baselines.json</code> (dev: random tree 0.0960, linear chain 0.4256); this is a manual check (no automatic check appears in the code)

There are two further "printed self-checks": `train_probe` prints the batch-mean L1 every 5 epochs; `run_control.py` prints `[检查] ... 选择性应为正` for each layer for human inspection.

3.5.10 Implementation Trade-offs and Limitations

- **First-subtoken pooling is the default** (`--pooling` defaults to `first`), and mean pooling serves only as the ablation; the two poolings share the same function, the difference being a single branch (`ix[0]` versus `h[b, ix].mean(0)`).
- The distance probe **has no bias and only a single rank = 128 matrix**; rank 128 is hard-coded as the default, and there is no rank sweep or selection procedure in the code; the MDL classifier, by contrast, uses the default `nn.Linear` (with bias).
- **Distance control is not conditioned on word type** (the `control_tasks.py` docstring explicitly states that this is a design choice): a whole tree is not a word-level label, and there is no standard way to condition on word type.

- The **random fake depth** is a **uniform integer on [0, max_depth]**, which does not match the distribution of true depths (concentrated in shallow layers); max_depth is determined by train alone.
- The MDL **code length is accumulated on train only**, with dev taking no part at all; each prefix has a fixed 5 epochs, with no early stopping and no hyperparameter search; the encoding probabilities of the last block come from the probe trained on only the first 90% of the data; the denominator of the compression ratio, uniform_bits, is the uniform code length of the full train set.
- An **unseen deprel in test is mapped to -1**, which only lowers the reference accuracy and does not affect the code length; on this data this situation occurs 0 times.
- A **single seed** (default 0) and a **single treebank** (the code reads only data/ud_gsdsimp/); data/ud_pud/zh_pud-ud-test.conllu exists in the data directory, but no script references it (it is not wired in).
- Tokenization **does not enable truncation**: the code has no truncation or chunking fallback, so a sentence exceeding the model's limit can only make the forward pass fail (the longest sentence in this treebank is 111 words, so this was never triggered in practice).
- The mean-pooling ablation covers **only the three monolingual models**: there is no mean-pooling group for mBERT (results/first_probe_bert-base-multilingual-cased_mean.json does not exist). summarize.py and plot_results.py only emit the corresponding rows and curves when the file exists and all layers are present, and models with missing data are skipped (plot_results.py prints a one-line skip notice for Figure 4(c)).
- Tables and figures are always **computed on the fly** from results/*.json (per-layer values, peak layer, minimum value, and runtimes are all computed at summarization time), and there are no hand-entered intermediate numbers in the code.

4 Results

4.1 RQ1: Existence and Location of the Syntactic Geometry

The full table of dev metrics for the three models across 13 extraction points is given in Table 1, the test metrics in Table 2, and the curves in Figure 1.

Table 1 dev (500 sentences) syntactic geometry metrics

Layer	BERT UUAS	BERT dSpr	BERT hSpr	BERT root	RoBERTa UUAS	RoBERTa dSpr	RoBERTa hSpr	RoBERTa root	MacBERT UUAS	MacBERT dSpr	MacBERT hSpr	MacBERT root
0	0.375	0.525	0.465	0.168	0.375	0.527	0.460	0.140	0.378	0.527	0.461	0.144
1	0.394	0.585	0.626	0.242	0.399	0.590	0.648	0.236	0.401	0.591	0.650	0.238
2	0.444	0.641	0.702	0.304	0.450	0.654	0.716	0.274	0.455	0.656	0.720	0.300
3	0.522	0.706	0.745	0.360	0.534	0.717	0.761	0.330	0.540	0.718	0.764	0.308
4	0.538	0.725	0.769	0.396	0.558	0.735	0.780	0.362	0.558	0.737	0.786	0.382
5	0.550	0.729	0.773	0.396	0.567	0.738	0.785	0.380	0.558	0.730	0.793	0.378
6	0.552	0.732	0.781	0.410	0.578	0.745	0.792	0.384	0.566	0.742	0.796	0.350
7	0.579	0.752	0.796	0.402	0.583	0.756	0.800	0.402	0.585	0.760	0.804	0.362
8	0.587	0.751	0.799	0.426	0.587	0.757	0.803	0.420	0.589	0.762	0.803	0.370
9	0.559	0.732	0.790	0.354	0.564	0.735	0.792	0.394	0.562	0.741	0.790	0.342
10	0.535	0.709	0.779	0.368	0.539	0.711	0.784	0.384	0.539	0.717	0.784	0.358
11	0.513	0.690	0.768	0.318	0.517	0.689	0.777	0.350	0.514	0.690	0.777	0.340
12	0.487	0.676	0.762	0.368	0.485	0.672	0.774	0.308	0.475	0.666	0.756	0.224

(dSpr = distance Spearman, hSpr = depth Spearman, root = root node accuracy. Baselines: random tree dev UUAS 0.096 / Spearman -0.001; linear chain 0.426 / 0.388.)

Table 2 test (500 sentences) UUAS / distance Spearman

Layer	BERT	RoBERTa	MacBERT
0	0.385 / 0.533	0.393 / 0.536	0.390 / 0.537
3	0.532 / 0.722	0.543 / 0.733	0.544 / 0.736
6	0.568 / 0.745	0.588 / 0.759	0.588 / 0.758
7	0.587 / 0.759	0.598 / 0.770	0.595 / 0.774
8	0.596 / 0.763	0.605 / 0.769	0.597 / 0.771
9	0.563 / 0.743	0.572 / 0.747	0.577 / 0.750
10	0.546 / 0.720	0.545 / 0.726	0.553 / 0.728
12	0.499 / 0.683	0.495 / 0.680	0.485 / 0.676

(The full 13 layers × 3 models table is in docs/results-tables.md.)

Finding 1: Chinese syntax is decodable from the representations, and the strength is substantial.

At the peak layer, dev UUAS is 0.587–0.589 and test 0.596–0.605, distance Spearman 0.75–0.77, depth Spearman about 0.80, and root accuracy 0.37–0.43. By comparison, with a random tree as the target, UUAS is only 0.096 (on a par with random guessing), and the linear chain baseline is 0.426 — lower than any model at layer 2 or above. In other words, there really is a tree structure in the model representations that can be recovered by a linear readout, and it is far better than any "linguistically irrelevant baseline tree."

Finding 2: The peak is at layer 8 in all cases (out of 13 extraction points, i.e. the 5th layer counting from the top), in a hump shape.

UUAS reaches its maximum at layer 8 for all three models; the distance Spearman peaks at layers 7–8 (for BERT, 0.752 at layer 7 and 0.751 at layer 8 are essentially on a par, while MacBERT/RoBERTa peak at layer 8); depth Spearman reaches 0.80 at layers 7–8. From then on the UUAS, the distance Spearman and the depth Spearman all decrease monotonically with layer: by layer 12 the UUAS falls to 0.475–0.487 and the depth Spearman to 0.756–0.774 (the root accuracy is a binary metric on 500 sentences and is noisy, fluctuating without following this trend).

Finding 3: The input layer (layer 0) already has a fairly high "surface" decodability.

UUAS at layer 0 already reaches 0.375–0.378, with distance Spearman 0.525–0.527, which comes from the character embeddings themselves and the positional encoding (word-order information) — this is exactly why control tests are necessary: high metrics are not necessarily syntax.

4.2 RQ2: The Influence of Pre-training Strategy and Model Provenance

(a) The three masking strategies converge by different routes.

Figure 1 (figures/fig1_syntax_geometry.png) plots the three monolingual model curves together, and the conclusion is quite clear-cut:

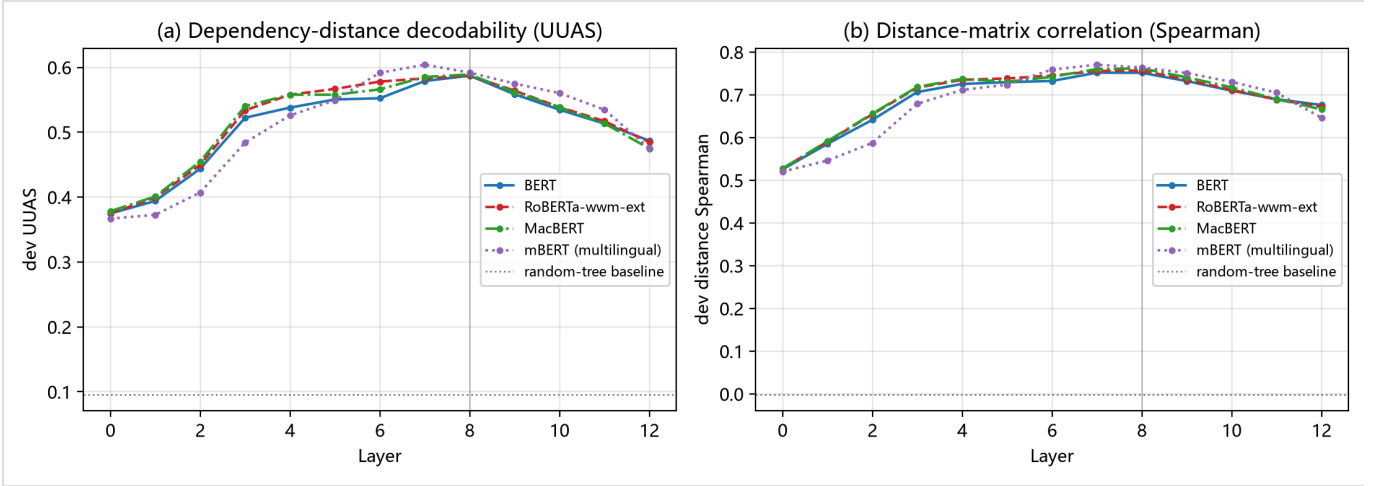


Figure 1 UUAS / distance Spearman curves for four models across 13 layers (with the random-tree baseline)

- **Same shape:** the three curves trace almost overlapping profiles across all layers, all being "layers 0–3 rising rapidly → layers 4–7 climbing slowly → layer 8 reaching the peak → declining thereafter."
- **Same peak layer:** the UUAS peak of all three models is at layer 8, with dev values 0.587 / 0.587 / 0.589 (differing by 0.2 percentage points) and test 0.596 / 0.605 / 0.597.
- **The strength differences are small and inconsistent:** the maximum gap across all layers is 2.6 percentage points (layer 6, RoBERTa 0.578 vs BERT 0.552); most layers differ by around 1 percentage point, and the ranking is not stable: layers 0–4 and 7–8 are slightly higher for MacBERT (0.4–2.0 percentage points), layers 5–6 are instead slightly higher for RoBERTa (1.6 and 2.6 percentage points), and the last layer is slightly higher for BERT. In MDL the ordering even flips: the lowest code length at layer 8 is BERT 157.1 kbit and RoBERTa 157.9 kbit.
- **The two kinds of targets, depth (hierarchical structure) and distance (dependency structure), give the same picture;** MacBERT's depth selectivity is slightly higher (see 4.3), and this is the only stably observable difference among the models.

The first answer to RQ2 is therefore: **the pre-training masking strategy (original MLM / wwm / error-correcting) changes downstream usability, not the syntactic geometry itself** — the three training objectives converge by different routes onto a syntactic geometry with the same layer, the same strength, and the same shape. It must be stated honestly that we ran only a single seed, so ordering differences on the scale of 1 percentage point cannot be distinguished from seed noise; what can safely be claimed is the robust conclusion that "the shape and the peak layer are consistent," not that "MacBERT is 1 point better than BERT."

(b) Multilingual mBERT is no weaker than the monolingual models, but its peak comes one layer earlier. We additionally ran mBERT under exactly the same pipeline, treebank, and hyperparameters (Table 6, Table 7).

Table 6 mBERT (bert-base-multilingual-cased) per-layer metrics

Layer	dev UUAS	dev dSpr	dev hSpr	test UUAS	test dSpr	depth Δ (dev)	distance Δ (dev)	distance control UUAS
0	0.367	0.520	0.473	0.379	0.526	-0.150	0.271	0.095
1	0.373	0.546	0.583	0.385	0.563	-0.111	0.280	0.093
2	0.407	0.587	0.643	0.417	0.600	-0.115	0.310	0.097
3	0.484	0.679	0.731	0.499	0.694	-0.047	0.387	0.097
4	0.526	0.711	0.753	0.546	0.726	-0.022	0.432	0.094
5	0.549	0.723	0.774	0.568	0.740	0.000	0.457	0.093
6	0.592	0.759	0.796	0.611	0.771	0.044	0.494	0.097

Layer	dev UUAS	dev dSpr	dev hSpr	test UUAS	test dSpr	depth Δ (dev)	distance Δ (dev)	distance control UUAS
7	0.604	0.770	0.806	0.619	0.778	0.072	0.502	0.102
8	0.592	0.764	0.808	0.599	0.768	0.095	0.492	0.100
9	0.575	0.751	0.800	0.589	0.756	0.102	0.476	0.099
10	0.560	0.730	0.794	0.576	0.736	0.098	0.466	0.095
11	0.535	0.705	0.782	0.551	0.712	0.096	0.442	0.092
12	0.475	0.646	0.759	0.487	0.656	0.104	0.376	0.099

Table 7 four-model peak comparison

Model	UUAS peak layer	dev UUAS	dev dSpr	test UUAS	test dSpr	dev hSpr peak	layer 8 depth Δ	MDL lowest layer	MDL lowest kbit
BERT	8	0.587	0.751	0.596	0.763	0.799 (layer 8)	+0.071	8	157.1
RoBERTa-wwm-ext	8	0.587	0.757	0.605	0.769	0.803 (layer 8)	+0.072	8	157.9
MacBERT	8	0.589	0.762	0.597	0.771	0.804 (layer 7)	+0.113	8	160.7
mBERT (multilingual)	7	0.604	0.770	0.619	0.778	0.808 (layer 8)	+0.095	7	158.3

mBERT is slightly higher than the three monolingual models on all four metrics, and its **UUAS peak comes earlier, at layer 7**: the distance selectivity (0.502 at layer 7 versus 0.492 at layer 8) and the minimum MDL code length (158.3 at layer 7 versus 159.6 kbit at layer 8) also fall in layer 7, so the three lines agree in direction; the only line that does not shift earlier is the peak of the depth Spearman (still layer 8, 0.808). We should state honestly that all three gaps are on the order of 1 percentage point and any single one of them lies within seed noise (limitation ① in Section 5): what deserves trust is the agreement in direction across the three lines, not the size of any individual gap. This result runs counter to the common impression ("multilingual models are weaker at Chinese"), but the two claims do not conflict: what Chi et al. (2020) found is that Chinese is encoded relatively weakly **within mBERT** compared with other languages (a cross-lingual relative comparison), whereas what we measure is **the absolute decodability across models** (Chinese monolingual models vs mBERT on Chinese). A model can be relatively weak within its own language family and still outperform a Chinese monolingual model trained specifically for the language — our data show that the latter holds.

The control tests rule out the explanation that "mBERT merely remembers more word forms": its layer-8 depth selectivity is +0.095, falling within the range of the three monolingual models (+0.071/+0.072/+0.113); the random-tree control stays at dev UUAS 0.092–0.102 across all layers, indistinguishable from chance; and the lower layers likewise show negative selectivity (layer 0 is −0.150), with the turn to positive also around layer 5. In other words, mBERT's mid-layer syntactic geometry is the same kind of thing as the monolingual models', only appearing earlier.

(c) Ablation: the conclusion does not depend on the Chinese-specific character↔word alignment.

After changing the word representation from "first subtoken" to "mean within the word" (Table 8):

Table 8 mean-pooling ablation (vs first-subtoken pooling)

Model	First-subtoken pooling peak	Mean pooling peak	Layer 8 depth Δ (first → mean)	Layer 8 distance Δ UUAS (first → mean)	MDL lowest kbit (first → mean)
BERT	layer 8, 0.587	layer 8, 0.617	+0.071 → +0.087	0.491 → 0.519	157.1 → 150.6
RoBERTa-wwm-ext	layer 8, 0.587	layer 8, 0.611	+0.072 → +0.083	0.489 → 0.512	157.9 → 151.3
MacBERT	layer 8, 0.589	layer 8, 0.613	+0.113 → +0.131	0.492 → 0.519	160.7 → 154.3

Four things hold at once (Figure 4): **the UUAS peak layer is entirely unchanged (still 8 for all three models)**; the absolute values shift up by about 3 percentage points overall (layer 8 dev UUAS 0.587 /

0.587 / 0.589 $\rightarrow$ 0.617 / 0.611 / 0.613); **both the depth and the distance selectivity shift up in step** (layer 8 depth Δ +0.071 / +0.072 / +0.113 $\rightarrow$ +0.087 / +0.083 / +0.131; layer 8 distance Δ UUAS 0.491 / 0.489 / 0.492 $\rightarrow$ 0.519 / 0.512 / 0.519); and **the minimum MDL code length shortens at the same time** (157.1 / 157.9 / 160.7 $\rightarrow$ 150.6 / 151.3 / 154.3 kbit) while its layer is unchanged. The key point is that the control side does not shift with it — the random-tree control keeps dev UUAS within 0.091–0.104 across all layers, indistinguishable from the 0.096 baseline, so the shift comes from the true structure being read more cleanly rather than from stronger word-form memorization. Under mean pooling the peak layer of the distance selectivity is still layer 7–8 (layer 8 for BERT and MacBERT; for RoBERTa layer 7 at 0.517 is marginally above layer 8 at 0.512, a 0.5-percentage-point difference on the noise scale). Mean pooling merely gives the probe a cleaner, less noisy word representation, and the location and shape of the syntactic geometry are unaffected. The core conclusion of this paper is therefore robust to the Chinese-specific alignment handling.

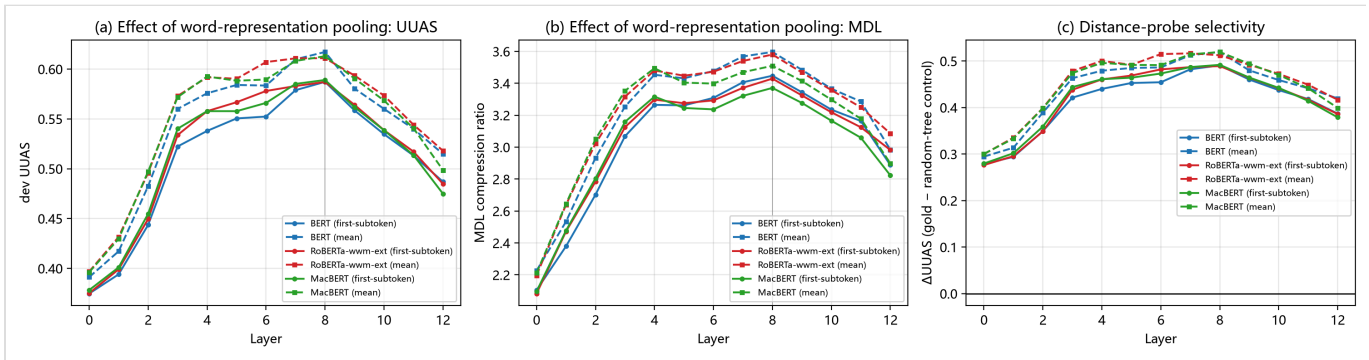


Figure 4 mean-pooling ablation (three models, first-subtoken solid vs mean dashed): (a) dev UUAS, (b) MDL compression ratio, (c) distance-probe selectivity

4.3 RQ3: Survival under Control Tests

(a) Depth control: low-layer decodability is mainly word-form memorization. Table 3 and Figure 2(a) give the per-layer selectivity.

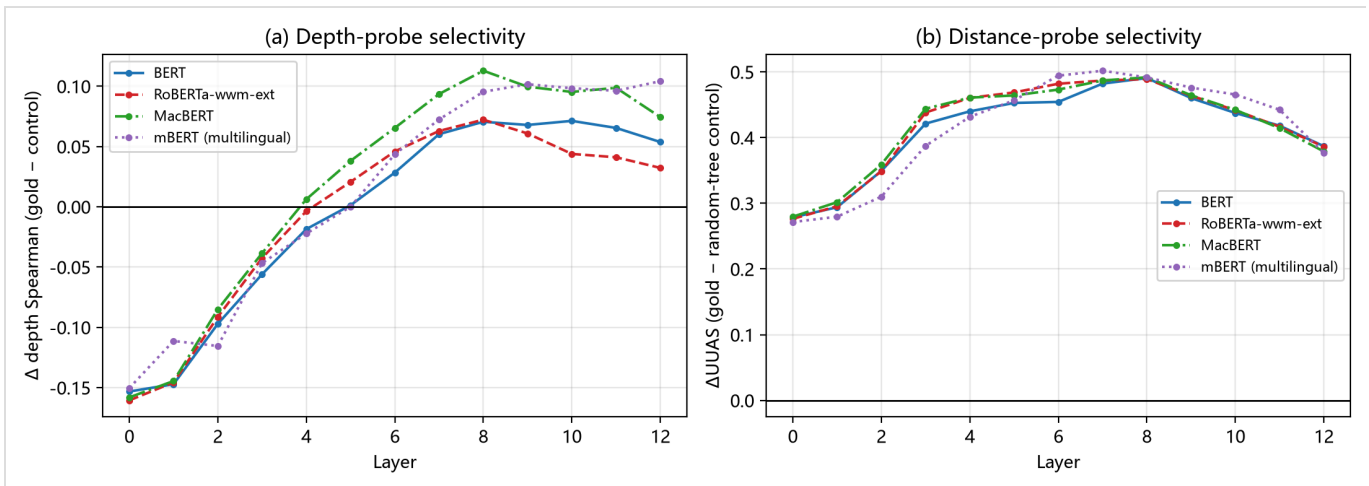


Figure 2 selectivity as a function of layer (depth / distance, four models)

Table 3 depth probe selectivity (dev, gold depth Spearman – control depth Spearman)

Layer	BERT Δ	BERT control	RoBERTa Δ	RoBERTa control	MacBERT Δ	MacBERT control
0	-0.153	0.619	-0.161	0.621	-0.158	0.619
1	-0.147	0.773	-0.146	0.794	-0.145	0.795
2	-0.097	0.799	-0.091	0.807	-0.085	0.805

Layer	BERT Δ	BERT control	RoBERTa Δ	RoBERTa control	MacBERT Δ	MacBERT control
3	-0.056	0.801	-0.043	0.803	-0.039	0.802
4	-0.019	0.788	-0.004	0.784	+0.006	0.780
5	+0.001	0.772	+0.021	0.764	+0.038	0.755
6	+0.028	0.753	+0.046	0.746	+0.065	0.731
7	+0.060	0.736	+0.063	0.737	+0.094	0.710
8	+0.071	0.729	+0.072	0.730	+0.113	0.690
9	+0.068	0.722	+0.061	0.731	+0.100	0.690
10	+0.071	0.708	+0.044	0.741	+0.095	0.689
11	+0.065	0.703	+0.041	0.736	+0.099	0.678
12	+0.054	0.709	+0.032	0.741	+0.074	0.682

The three models show the same pattern. At the very bottom, the control task's score is **higher** than the real task's (at layer 0 the control depth Spearman is 0.62 versus gold 0.46, with selectivity around -0.15): this shows that the thing most easily read out of the representation at that point is "what this word is" (word-type identity), and the random fake depth happens to assign "the same value to every occurrence of the same word type," which word-type identity can solve perfectly; a true tree depth, by contrast, requires computing structure. The control score reaches a peak of 0.77–0.81 at layers 1–3 and then declines. Selectivity turns from negative to positive at layers 4–5, then rises to **coincide with the syntactic peak at layer 8** (BERT +0.071, RoBERTa +0.072, MacBERT +0.113), and falls back again with layer thereafter.

The explanatory force of this result lies in the following: if the "syntactic peak" at layer 8 were merely a by-product of word-form information, selectivity should be most negative at layer 8, or at least not peak there; in fact, layer-8 selectivity and the probe metrics peak at the same time. Therefore the claim that **"a syntactic geometry exists in the middle layers" survives the control test**, while the claim that "syntax can already be decoded in the lower layers" is falsified — the high metrics of the lower layers must be discounted. MacBERT's depth selectivity is clearly higher than the other two models (0.113 versus 0.071/0.072 at layer 8); this is the only stably observable difference among the three models, and it may be related to its error-correcting objective (replacing masked words with synonyms, pushing the model to rely more on context rather than vocabulary identity), but with a single seed it should not be over-interpreted.

(b) Distance control: random structure cannot enter the representation. Table 4 and Figure 2(b): for probes targeting a fixed random tree, dev UUAS lies between 0.093 and 0.104 across all layers, indistinguishable from the random-tree baseline of 0.096; dev Spearman stays within $|p| \leq 0.01$ for all three models. Conversely, the selectivity of the true-tree probe rises monotonically from 0.279 at layer 0 to 0.491 at layer 8 (0.489–0.492 for the three models), and then falls back to 0.386–0.387. This conclusion does not change with the pooling method: switching to the within-word mean (Figure 4(c)) leaves the random-tree control at dev UUAS 0.091–0.104 (baseline 0.096) with $|p| \leq 0.011$ across all layers, while the layer-8 selectivity rises to 0.512–0.519 (0.489–0.492 for first-subtoken). The control scores do not move and the true-tree scores move up — direct evidence that the shift comes from a cleaner true-structure signal.

It is worth noting that a "negative control" is itself very convincing: the random-tree control has a positive Spearman on **train** (0.167–0.170 at layer 8 for the three models) but 0 on dev — the probe did indeed memorize the random trees of the training sentences, but this memory does not transfer at all. This shows that a random-tree target can only be solved by word-by-word memorization, and that the representation does not contain, and cannot provide, the geometry of such a random structure;

conversely, the selectivity of the true tree is strongly positive across all layers, showing that what the model representation provides is true structural information.

Table 4 distance probe selectivity (dev, gold UAS – random-tree control UAS)

Layer	BERT Δ	BERT control	RoBERTa Δ	RoBERTa control	MacBERT Δ	MacBERT control
0	0.279	0.096	0.276	0.099	0.280	0.099
2	0.349	0.094	0.349	0.101	0.359	0.096
4	0.440	0.098	0.460	0.097	0.461	0.097
6	0.454	0.098	0.482	0.096	0.473	0.093
8	0.491	0.096	0.489	0.098	0.492	0.097
10	0.437	0.097	0.442	0.097	0.442	0.096
12	0.387	0.100	0.386	0.099	0.379	0.096

(c) MDL: independently reproducing layer 8 with bits. Table 5 and Figure 3.

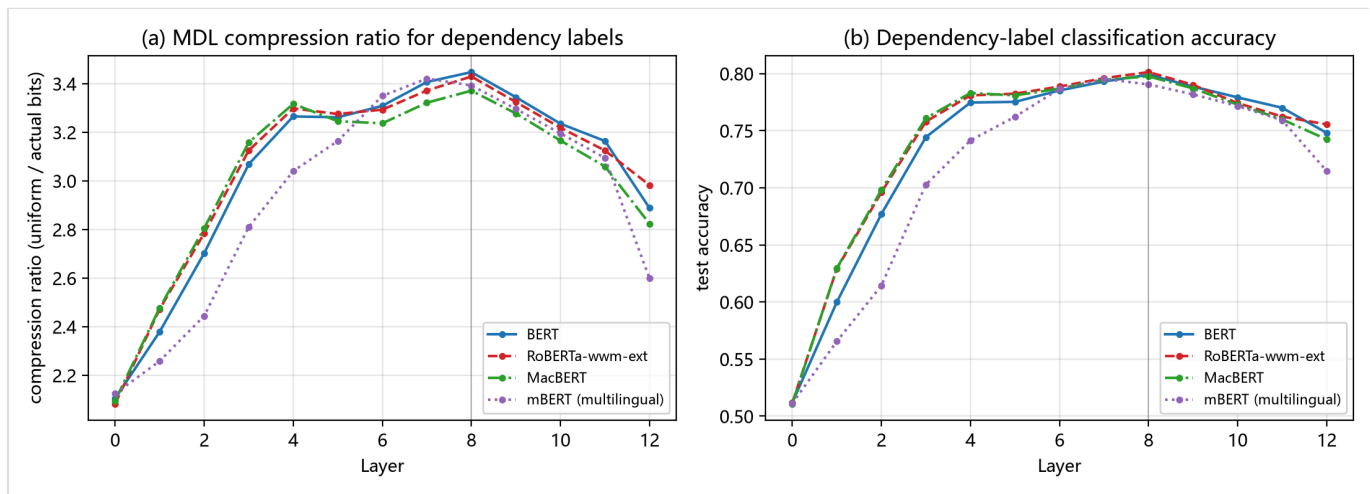


Figure 3 MDL compression ratio and dependency relation classification accuracy (four models)

Table 5 MDL: online code length for the 45 dependency relation labels (train, lower is better)

Layer	BERT kbit	BERT compression	BERT acc	RoBERTa kbit	RoBERTa compression	RoBERTa acc	MacBERT kbit	MacBERT compression	MacBERT acc
0	257.4	2.10	0.512	260.0	2.08	0.512	258.4	2.10	0.511
1	227.6	2.38	0.600	219.1	2.47	0.629	218.6	2.48	0.630
2	200.4	2.70	0.677	194.5	2.78	0.696	193.1	2.81	0.698
3	176.5	3.07	0.744	173.3	3.13	0.758	171.5	3.16	0.761
4	165.9	3.27	0.775	164.2	3.30	0.781	163.3	3.32	0.783
5	166.1	3.26	0.775	165.4	3.28	0.782	166.9	3.25	0.781
6	163.7	3.31	0.785	164.5	3.29	0.789	167.3	3.24	0.787
7	159.0	3.41	0.793	160.6	3.37	0.796	163.1	3.32	0.795
8	157.1	3.45	0.799	157.9	3.43	0.801	160.7	3.37	0.798
9	161.9	3.34	0.789	162.9	3.32	0.790	165.3	3.28	0.787
10	167.4	3.24	0.779	168.3	3.22	0.774	171.1	3.17	0.773
11	171.2	3.16	0.770	173.3	3.12	0.762	177.0	3.06	0.760
12	187.5	2.89	0.748	181.6	2.98	0.755	191.8	2.82	0.742

(Uniform encoding 541.6 kbit; compression ratio = uniform code length / actual code length.)

From layer 0 to layer 8 the compression ratio rises from 2.10× to 3.45× and the test accuracy from 0.512 to 0.799; along the way the step from layer 4 to layer 5 shows one very small increase in all three models (code length +0.2 / +1.2 / +3.6 kbit, plus a further +0.4 kbit from layer 5 to layer 6 for MacBERT), which does not change the overall trend. From layer 8 to layer 12 the code length then rises

strictly monotonically back to $2.82\text{--}2.98\times$ and accuracy $0.742\text{--}0.755$. **The minimum code length of all three models falls at layer 8**, coinciding with the probe peak and the selectivity peak.

The significance of MDL is that it uses a different ruler: probe metrics measure "whether it can be read out," while MDL counts the probe's complexity into the cost and measures "whether the information has been placed where it can be read out cheaply and generalize to new data." The two independently point to layer 8, so the objection of "false positives caused by excessive probe capacity" does not hold here.

4.4 Summary: Convergence of the Three Lines of Evidence

Line of evidence	Metric	Peak layer (consistent across the three monolingual models)	Value at the peak (BERT / RoBERTa / MacBERT)
Structural probe (distance)	dev UUAS	layer 8	0.587 / 0.587 / 0.589
Structural probe (depth)	dev depth Spearman	layers 7–8	0.799 / 0.803 / 0.804
Control task selectivity (depth)	Δ Spearman	layer 8	0.071 / 0.072 / 0.113
Control-task selectivity (distance)	Δ UUAS	layers 7–8	0.491 / 0.489 / 0.492 (first-subtoken); 0.519 / 0.512 / 0.519 (mean)
MDL	encoding bits (minimum)	layer 8	157.1 / 157.9 / 160.7 kbit

Conclusions of the four robustness checks:

- **Changing the training objective** (LM / wwm / error-correcting): same peak layer, same shape, strength difference ≤ 2.6 percentage points (4.2a).
- **Changing the model provenance** (multilingual mBERT): the peak comes earlier, at layer 7, and the strength is slightly higher (dev 0.604, test 0.619), with selectivity and MDL pointing the same way (4.2b).
- **Changing the word-representation pooling** (first subtoken $\rightarrow$ within-word mean): the UUAS peak layer is unchanged, the absolute values shift up by about 3 percentage points, and both the depth and the distance selectivity shift up in step (layer 8 distance Δ UUAS $0.489\text{--}0.492 \rightarrow 0.512\text{--}0.519$), while the control scores stay put (Figure 4(c), 4.2c).
- **Changing the target structure** (true tree $\rightarrow$ random tree / random fake depth): the advantage of the true structure still holds after testing, and is strongest at layer 8 (4.3).

5 Discussion

Why is the peak at layer 8 rather than the top layer? The way the representation is used provides the most natural explanation. The metrics of layers 9–12 decline collectively (UUAS dropping from 0.587 to around 0.48, and the MDL code length rising by 20%), and layer 12 happens to be the layer closest to the output: the pre-training objective requires it to "spread out" the representation again into a form convenient for vocabulary prediction, i.e. it tilts towards lexical-semantic specialization. Syntax is retained as a product of the middle layers, and at the top serves the reconstruction task. This is consistent with the division of labour across layers observed in English models: Tenney et al. (2019) find that the layer order roughly reproduces the classical NLP pipeline (POS tagging $\rightarrow$ parsing $\rightarrow$ NER $\rightarrow$ semantic roles $\rightarrow$ coreference) and that performance drops in the last 1–2 layers, indicating that the information for an intermediate step such as syntax is concentrated in the middle layers and gives way to semantics and reconstruction towards the top. This division of labour is therefore not specific to Chinese or to any particular training strategy.

Relation to the cross-lingual evidence. Chi et al. (2020) found, on multilingual mBERT, that Chinese is one of the more poorly encoded languages — this is a cross-lingual relative comparison **within** mBERT. This paper additionally ran mBERT under the same pipeline and obtained a result that points in the opposite direction but does not conflict: mBERT's absolute decodability on Chinese (dev peak 0.604, test 0.619) is **no lower than** that of the three Chinese monolingual models trained specifically for the language (dev 0.587–0.589, test 0.596–0.605), and its peak layer is one layer earlier. A model can be relatively weak within its own language family while still outperforming monolingual models; our data support the latter reading. It should be noted that absolute numbers across papers are strongly affected by the treebank version, the alignment method, and the hyperparameters, so all comparisons in this paper are confined to the same pipeline, the same treebank, and the same set of hyperparameters, and are not made directly against numbers in the literature.

Methodological conclusion (of general significance for Chinese probe research). The negative selectivity of the lower layers is the point most worth emphasizing: if a Chinese probing study does not run a control task, it is highly likely to overestimate the decodability of syntax at layers 0–4, because what is most easily read out there is in fact character/word identity information. Our data give a concrete magnitude for this warning — the "apparent syntactic decodability" at layer 0 is as high as 0.375 UUAS, of which 0.096 is a level any random structure can reach, while selectivity shows that this layer is in fact negative.

Limitations. ① A single seed, with no multi-seed statistical testing, so ordering differences on the scale of 1 percentage point (the between-model comparison of Section 4.2b) should not be taken as a conclusion; ② a single treebank (UD Chinese-GSDSimp, Wikipedia register, 500 dev/test sentences), so absolute values are sensitive to the treebank and the annotation scheme; ③ the probing paradigm can only show that information is "decodable," not that the model "causally uses" it at inference time; ④ larger scales and generative architectures (the Qwen series) are not covered, so whether the "peak at layers 7–8" drifts with scale remains unknown; ⑤ the mean-pooling ablation covers only the three monolingual models, and mBERT has no pooling control; ⑥ the evaluation counts punctuation words in the dependency graph (3.5.4), so the absolute UUAS values in this paper are higher than the common convention of "excluding punctuation," and numbers should not be compared digit by digit across papers (all comparisons in this paper are within the same pipeline). (The worry that "the Chinese-specific first-subtoken pooling affects the conclusion" has been ruled out by the ablation in 4.2c, and is no longer listed as a limitation.)

6 Conclusion

This paper applied Hewitt & Manning structural probes, two-way control tasks, and an MDL test to four models of the same size (three Chinese monolingual PLMs and multilingual mBERT) on UD Chinese-GSDSimp. Three independent lines of evidence (decodability, selectivity, code length) consistently locate the peak of Chinese syntactic geometry at layer 8 (layer 7 for mBERT); this geometry is highly robust to the pre-training masking strategy (same peak layer, all-layer gap ≤ 2.6 percentage points), and does not depend on the Chinese-specific character $\leftrightarrow$ word pooling (under mean pooling the peak layer and shape are unchanged, with the absolute values shifted up by about 3 percentage points); the surface decodability of the lower layers is mostly attributed to word-form memorization after the control test, while the mid-layer syntactic signal survives the test. Multilingual mBERT is no weaker than the monolingual Chinese models within this pipeline. These results support a moderate but clear conclusion: Chinese PLMs do encode dependency syntax in their own representational geometry, and

the location and strength of this encoding are determined mainly by "the role of the layer in the computation," not by the specific pre-training objective or the language coverage.

Future work, in order of priority: ① multi-seed repetition, to turn 1-percentage-point-scale model differences into credible statistical statements; ② testing the hypothesis that "the peak layer drifts with scale" on larger-scale and generative models (the Qwen series, the Zh-Pythia ladder); ③ connecting phenomenon-level probes (把 / 被 constructions, aspect markers) to the categories where model behaviour fails in CLiMP / SLING, to see whether "whether it is in the representation" and "whether it is used in behaviour" dissociate.

References

(Bibliographic details were verified against primary sources such as the ACL Anthology / publisher pages / the official UD repository / arXiv: the first round of verification was done on 2026-10-02, and the RoBERTa entry was added and verified on 2026-10-04; all page numbers come from primary-page metadata, with no estimated values. Hugging Face model cards are unreachable from this machine's network, so model provenance was instead verified from the official GitHub repositories.)

- Devlin, Chang, Lee & Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019, 4171–4186. <https://aclanthology.org/N19-1423/> (the companion paper for bert-base-chinese)
- Liu, Ott, Goyal, Du, Joshi, Chen, Levy, Lewis, Zettlemoyer & Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019. <https://arxiv.org/abs/1907.11692> (the pre-training recipe followed by hf1/chinese-roberta-wwm-ext)
- Hewitt & Manning. A Structural Probe for Finding Syntax in Word Representations. NAACL-HLT 2019, 4129–4138. <https://aclanthology.org/N19-1419/>
- Hewitt & Liang. Designing and Interpreting Probes with Control Tasks. EMNLP-IJCNLP 2019, 2733–2743. <https://aclanthology.org/D19-1275/>
- Tenney, Das & Pavlick. BERT Rediscovered the Classical NLP Pipeline. ACL 2019, 4593–4601. <https://aclanthology.org/P19-1452/>
- Clark, Khandelwal, Levy & Manning. What Does BERT Look At? An Analysis of BERT's Attention. BlackboxNLP @ ACL 2019, 276–286. <https://aclanthology.org/W19-4828/>
- Voita & Titov. Information-Theoretic Probing with Minimum Description Length. EMNLP 2020, 183–196. <https://aclanthology.org/2020.emnlp-main.14/>
- Pimentel, Cotterell, Teufel & Blunsom. Information-Theoretic Probing for Linguistic Structure. ACL 2020, 4609–4622. <https://aclanthology.org/2020.acl-main.420/>
- Pimentel, Saphra, Teufel, Cotterell & Blunsom. Pareto Probing: Trading Off Accuracy for Complexity. EMNLP 2020, 3138–3153. arXiv:2010.02180
- Chi, Hewitt & Manning. Finding Universal Grammatical Relations in Multilingual BERT. ACL 2020, 5564–5577. <https://aclanthology.org/2020.acl-main.493/>
- Kulmizev, Ravishankar, Abdou & Nivre. Do Neural Language Models Show Preferences for Syntactic Formalisms? ACL 2020, 4077–4091. <https://aclanthology.org/2020.acl-main.375/>
- Xiang, Yang, Li, Warstadt & Kann. CLiMP: A Benchmark for Chinese Language Model Evaluation. EACL 2021, 2784–2790. <https://aclanthology.org/2021.eacl-main.242/>
- Song, Krishna, Bhatt & Iyyer. SLING: Sino Linguistic Evaluation of Large Language Models. EMNLP 2022, 4606–4634. <https://aclanthology.org/2022.emnlp-main.305/>

- Liu, Shen, Zhu, Xu, Qian, Song, Zhang, Tang, Zhang, Yang, Wang & Hu. A Systematic Assessment of Language Models with Linguistic Minimal Pairs in Chinese (the ZhoBLiMP dataset). TACL, Vol. 14, 2026, 755–771. <https://aclanthology.org/2026.tacl-1.34/> ; preprint arXiv:2411.06096
- Zheng & Liu. What does Chinese BERT learn about syntactic knowledge? PeerJ Computer Science 9:e1478, 2023. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10403162/>
- Cui, Che, Liu, Qin & Yang. Pre-Training with Whole Word Masking for Chinese BERT. IEEE/ACM TASLP, Vol. 29, 2021, 3504–3514. <https://doi.org/10.1109/TASLP.2021.3124365> (the source of hf1/chinese-roberta-wwm-ext)
- Cui, Che, Liu, Qin, Wang & Hu. Revisiting Pre-Trained Models for Chinese Natural Language Processing (MacBERT). Findings of EMNLP 2020, 657–668. <https://aclanthology.org/2020.findings-emnlp.58/> (the source of hf1/chinese-macbert-base)
- Rogers, Kovaleva & Rumshisky. A Primer in BERTology: What We Know About How BERT Works. TACL, Vol. 8, 2020, 842–866. <https://aclanthology.org/2020.tacl-1.54/>
- Nivre et al. Universal Dependencies v1: A Multilingual Treebank Collection. LREC 2016, 1659–1666. <https://aclanthology.org/L16-1262/>
- The UD Chinese-GSDSimp treebank has no standalone paper and is cited together with the UD data release (included since UD v2.5; contributors Peng Qi, Koichi Yasuoka; licence CC BY-SA 4.0). https://universaldependencies.org/treebanks/zh_gsdsimp/index.html

List of Figures and Tables

- Figure 1: four models × 13 layers UUAS / distance Spearman curves (including the random-tree baseline) — [figures/fig1_syntax_geometry.png](#)
- Figure 2: selectivity as a function of layer (depth / distance, four models) — [figures/fig2_selectivity.png](#)
- Figure 3: MDL compression ratio and dependency relation classification accuracy (four models) — [figures/fig3_md1.png](#)
- Figure 4: mean-pooling ablation (three models, first-subtoken solid vs mean dashed) — (a) dev UUAS, (b) MDL compression ratio, (c) distance-probe selectivity — [figures/fig4_pooling_ablation.png](#)
- Table 1: full dev per-layer metrics (three monolingual models); Table 2: per-layer test UUAS/distance Spearman; Table 3: depth selectivity; Table 4: distance selectivity; Table 5: MDL; Tables 6–7: mBERT per-layer metrics and the four-model peak comparison; Table 8: mean pooling ablation (all numbers are generated automatically by `src/summarize.py` from `results/*.json`, see `docs/results-tables.md`)